

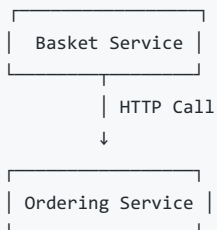
RabbitMQ + MassTransit - Complete Guide Using Your Project

Table of Contents

1. [The Problem RabbitMQ Solves](#)
2. [RabbitMQ Configuration in Docker Compose](#)
3. [MassTransit Implementation in Services](#)
4. [Event Definitions \(Shared Library\)](#)
5. [Event Flow - Complete Example](#)
6. [Queue Architecture](#)
7. [Key Concepts](#)
8. [Summary](#)

The Problem RabbitMQ Solves

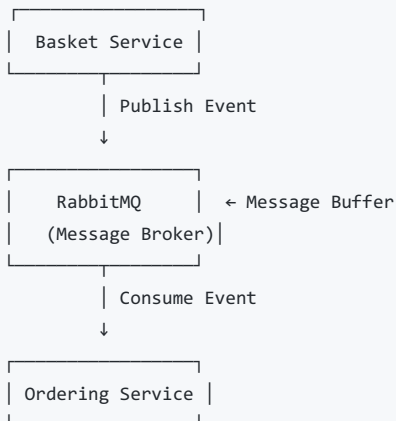
Without RabbitMQ (Direct HTTP Calls)



PROBLEMS:

- If Ordering Service is down → Basket Service fails
- If Ordering Service is slow → Basket Service waits (blocks user)
- If 1000 users checkout → Ordering Service gets overwhelmed
- Tight coupling between services
- No message buffering

With RabbitMQ (Message Broker)



BENEFITS:

- Services are decoupled (don't know about each other)
- Asynchronous communication (no blocking)
- RabbitMQ buffers messages (handles traffic spikes)
- Messages not lost if service is down
- Scalable (multiple consumers can process messages)

RabbitMQ Configuration in Docker Compose

docker-compose.yml (Lines 17-18)

```
rabbitmq:
  image: rabbitmq:3-management-alpine
```

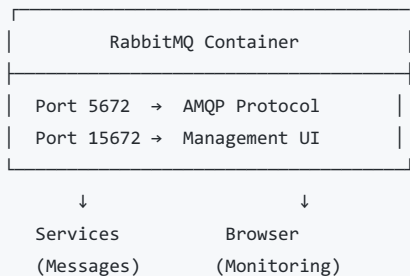
What this does:

- Defines RabbitMQ service
- Uses `rabbitmq:3-management-alpine` image (includes management UI)
- Alpine = lightweight Linux distribution

docker-compose.override.yml (Lines 65-70)

```
rabbitmq:
  container_name: rabbitmq
  restart: always
  ports:
    - "5672:5672"      # AMQP protocol (message communication)
    - "15672:15672"    # Management UI (web interface)
```

Port Configuration Diagram



Access Management UI:

- Open browser: `http://localhost:15672`
- Default credentials: `guest / guest`

Environment Variables for Services

Basket Service Configuration

```
basket:
  environment:
    - "EventBusSettings__HostAddress=amqp://guest:guest@rabbitmq:5672/"
  depends_on:
    - basketdb
    - rabbitmq
```

Ordering Service Configuration

```
ordering:
  environment:
    - "EventBusSettings__HostAddress=amqp://guest:guest@rabbitmq:5672/"
  depends_on:
    - orderdb
    - rabbitmq
```

Payment Service Configuration

```

payment:
  environment:
    - "EventBusSettings__HostAddress=amqp://guest:guest@rabbitmq:5672/"
  depends_on:
    - rabbitmq

```

Connection String Breakdown

```

amqp://guest:guest@rabbitmq:5672/
  ↓      ↓      ↓      ↓
protocol user password host  port

```

Note: rabbitmq is the Docker container name (Docker internal DNS)

MassTransit Implementation in Services

MassTransit is a .NET library that makes it easy to work with message brokers like RabbitMQ.

NuGet Packages Required

Basket.Application/Basket.Application.csproj (Line 11)

```
<PackageReference Include="MassTransit.RabbitMQ" Version="8.5.2" />
```

Ordering.Application/Ordering.Application.csproj (Line 16)

```
<PackageReference Include="MassTransit.RabbitMQ" Version="8.5.2" />
```

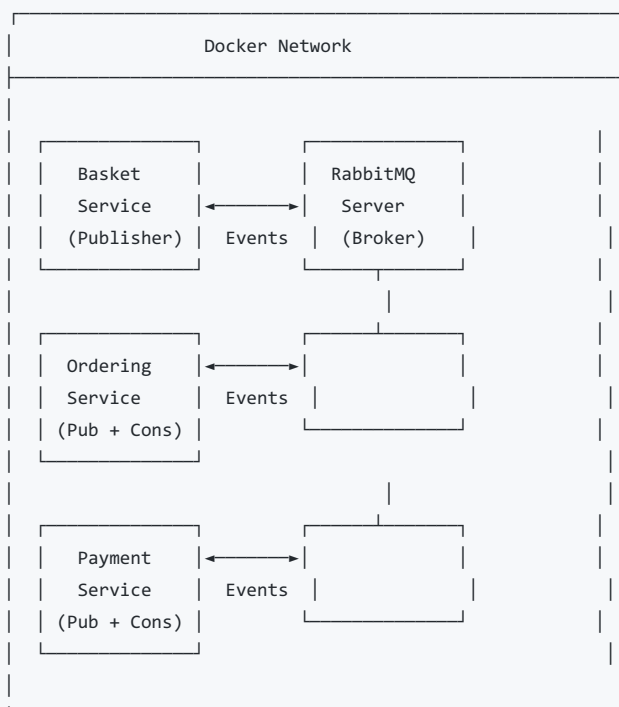
Payment/Payment.csproj (Lines 15-16)

```

<PackageReference Include="MassTransit" Version="8.5.2" />
<PackageReference Include="MassTransit.RabbitMQ" Version="8.5.2" />

```

Service Architecture Diagram



Configure MassTransit in Program.cs

Basket Service (Publisher Only)

File: Basket.API/Program.cs (Lines 72-80)

```
//Add mass Transit
builder.Services.AddMassTransit(config =>
{
    config.UsingRabbitMq((ct, cfg) =>
    {
        cfg.Host(builder.Configuration["EventBusSettings:HostAddress"]);
    });
});
```

What this does:

- Registers MassTransit services
- Configures RabbitMQ as the message broker
- Reads connection string from configuration

Ordering Service (Publisher + Consumer)

File: Ordering.API/Program.cs (Lines 26-54)

```
//Mass Transit
builder.Services.AddMassTransit(config =>
{
    //Mark as Consumer
    config.AddConsumer<BasketOrderingConsumer>();
    config.AddConsumer<PaymentCompletedConsumer>();
    config.AddConsumer<PaymentFailedConsumer>();

    config.UsingRabbitMq((ctx, cfg) =>
    {
        cfg.Host(builder.Configuration["EventBusSettings:HostAddress"]);
        cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));

        //provide the queue name with consumer settings
        cfg.ReceiveEndpoint(EventBusConstant.BasketCheckoutQueue, c =>
        {
            cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));
            c.ConfigureConsumer<BasketOrderingConsumer>(ctx);
        });

        cfg.ReceiveEndpoint(EventBusConstant.PaymentCompleteQueue, c =>
        {
            cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));
            c.ConfigureConsumer<PaymentCompletedConsumer>(ctx);
        });

        cfg.ReceiveEndpoint(EventBusConstant.PaymentFailedQueue, c =>
        {
            cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));
            c.ConfigureConsumer<PaymentFailedConsumer>(ctx);
        });
    });
});
```

Configuration Breakdown:

MassTransit Configuration Steps

1. Register Consumers
`config.AddConsumer<T>()`
↓
2. Configure RabbitMQ
`config.UsingRabbitMq()`
↓
3. Add Retry Policy
`cfg.UseRetry(r => r.Interval(5, 10s))`
↓
4. Create Queues
`cfg.ReceiveEndpoint(queueName)`
↓
5. Bind Consumers to Queues
`c.ConfigureConsumer<T>(ctx)`

Payment Service (Consumer + Publisher)

File: `Payment/Program.cs` (Lines 13-27)

```
// MassTransit
builder.Services.AddMassTransit(config =>
{
    config.AddConsumer<OrderCreatedConsumer>();
    config.UsingRabbitMq((ctx, cfg) =>
    {
        cfg.Host(builder.Configuration["EventBusSettings:HostAddress"]);
        cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));

        cfg.ReceiveEndpoint(EventBusConstant.OrderCreatedQueue, c =>
        {
            c.UseMessageRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));
            c.ConfigureConsumer<OrderCreatedConsumer>(ctx);
        });
    });
});
```

Event Definitions (Shared Library)

EventBusConstant (Queue Names)

File: `Infrastructure/EventBus.Messages/Common/EventBusConstant.cs`

```
public class EventBusConstant
{
    public const string BasketCheckoutQueue = "basketcheckout-queue";
    public const string OrderCreatedQueue = "ordercreated-queue";
    public const string PaymentCompleteQueue = "paymentcompleted-queue";
    public const string PaymentFailedQueue = "paymentfailed-queue";
}
```

Why constants?:

- Centralized queue name management
- Avoids typos
- Easy to rename queues in one place

Event Classes Structure



Event Class Definitions

BasketCheckoutEvent

```
public record BasketCheckoutEvent : BaseIntegrationEvent
{
    public string UserName { get; set; }
    public decimal TotalPrice { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string EmailAddress { get; set; }
    public string AddressLine { get; set; }
    public string Country { get; set; }
    public string State { get; set; }
    public string ZipCode { get; set; }
    public string CardName { get; set; }
    public string CardNumber { get; set; }
    public string Expiration { get; set; }
    public string CVV { get; set; }
    public int PaymentMethod { get; set; }
}
```

OrderCreatedEvent

```
public record OrderCreatedEvent : BaseIntegrationEvent
{
    public int Id { get; set; }
    public string UserName { get; set; }
    public decimal TotalPrice { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string EmailAddress { get; set; }
    public string AddressLine { get; set; }
    public string Country { get; set; }
    public string State { get; set; }
    public string ZipCode { get; set; }
    public string CardName { get; set; }
    public string CardNumber { get; set; }
    public string Expiration { get; set; }
    public string CVV { get; set; }
    public int PaymentMethod { get; set; }
}
```

PaymentCompletedEvent

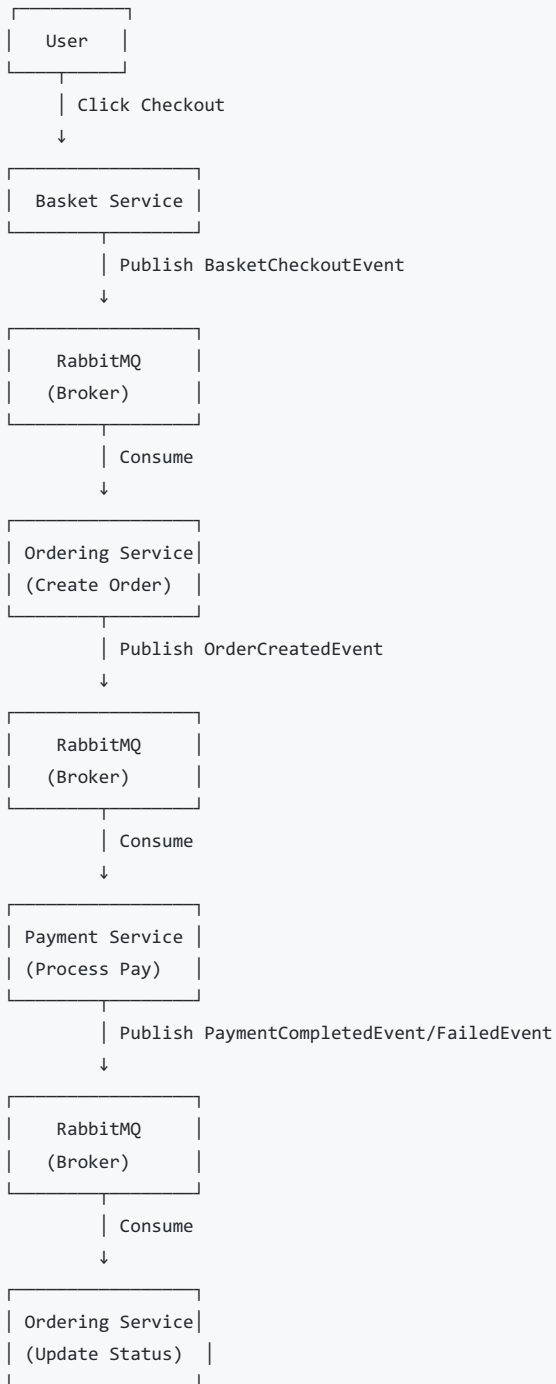
```
public record PaymentCompletedEvent : BaseIntegrationEvent
{
    public int OrderId { get; set; }
    public string UserName { get; set; }
    public decimal TotalPrice { get; set; }
    public Guid CorrelationId { get; set; }
}
```

PaymentFailedEvent

```
public record PaymentFailedEvent : BaseIntegrationEvent
{
    public int OrderId { get; set; }
    public string UserName { get; set; }
    public string Reason { get; set; }
    public Guid CorrelationId { get; set; }
}
```

Event Flow - Complete Example

The Complete Checkout Flow Diagram



Step-by-Step Code Flow

Step 1: User Checkout (Basket Service)

File: `Basket.Application/Handlers/BasketCheckoutHandler.cs` (Lines 24-40)


```

public async Task<Unit> Handle(BasketCheckoutCommand request, CancellationToken cancellationToken)
{
    var basketDto = request.BasketCheckoutDto;

    // 1. Get the basket using MediatR
    var basketResponse = await _mediator.Send(new GetBasketByUserNameQuery(basketDto.UserName), cancellationToken);

    if (basketResponse is null || !basketResponse.Items.Any())
    {
        throw new InvalidOperationException("Basket not found or empty");
    }

    // 2. Convert to entity
    var basket = basketResponse.ToEntity();

    // 3. Map to event
    var evt = basketDto.ToBasketCheckoutEvent(basket);

    // 4. Publish event to RabbitMQ
    _logger.LogInformation("Publishing BasketCheckoutEvent For {User}", basket.UserName);
    await _publishEndpoint.Publish(evt, cancellationToken);

    // 5. Delete the basket
    await _mediator.Send(new DeleteBasketByUserNameCommand(basket.UserName), cancellationToken);

    return Unit.Value;
}

```

Key Operation: `await _publishEndpoint.Publish(evt, cancellationToken);`

- Sends `BasketCheckoutEvent` to RabbitMQ
- `MassTransit` handles the connection and serialization

Step 2: Ordering Service Consumes Basket Checkout

File: `Ordering.Application/EventBusConsumer/BasketOrderingConsumer.cs`

```

public class BasketOrderingConsumer : IConsumer<BasketCheckoutEvent>
{
    private readonly ICommandHandler<CreateOrderCommand, int> _createOrderHandler;
    private readonly ILogger<BasketOrderingConsumer> _logger;

    public BasketOrderingConsumer(ICommandHandler<CreateOrderCommand, int> createOrderHandler, ILogger<BasketOrderingConsumer> logger)
    {
        _createOrderHandler = createOrderHandler;
        _logger = logger;
    }

    public async Task Consume(ConsumeContext<BasketCheckoutEvent> context)
    {
        using var scope = _logger.BeginScope("Consuming Basket Checkout Event for {CorrelationId}", context.Message.CorrelationId);

        // Convert event to command
        var command = context.Message.ToCheckoutOrderCommand();

        // Create order in database
        var orderId = await _createOrderHandler.Handle(command, context.CancellationToken);

        _logger.LogInformation("Basket checkout Event Completed Successfully !! OrderId: {OrderId}", orderId);
    }
}

```

What happens:

1. `MassTransit` automatically calls this when message arrives in queue

2. `ConsumeContext<BasketCheckoutEvent>` contains the message
3. Converts event to command
4. Calls command handler to create order
5. Order is saved to database

Step 3: Ordering Service Publishes Order Created

After creating order, Ordering service publishes `OrderCreatedEvent` (this would be in the order creation handler, following the same pattern as Basket checkout).

Step 4: Payment Service Consumes Order Created

File: `Payment/EventBusConsumer/OrderCreatedConsumer.cs`

```
public class OrderCreatedConsumer : IConsumer<OrderCreatedEvent>
{
    private readonly IPublishEndpoint _publishEndpoint;
    private readonly ILogger<OrderCreatedConsumer> _logger;

    public OrderCreatedConsumer(IPublishEndpoint publishEndpoint, ILogger<OrderCreatedConsumer> logger)
    {
        _publishEndpoint = publishEndpoint;
        _logger = logger;
    }

    public async Task Consume(ConsumeContext<OrderCreatedEvent> context)
    {
        var message = context.Message;
        _logger.LogInformation("Processing Payment for Order Id : {@OrderId} ", message.Id);

        // Simulate Payment Processing
        await Task.Delay(1000);

        if (message.TotalPrice > 0)
        {
            // Payment Success
            var completedEvent = new PaymentCompletedEvent
            {
                OrderId = message.Id,
                CorrelationId = context.CorrelationId.Value,
            };
            await _publishEndpoint.Publish(completedEvent);
            _logger.LogInformation("Payment success for Order Id : {@OrderId} and Correlation Id : {@CorrelationId}", message.Id, m
        }
        else
        {
            // Payment Failed
            var failedEvent = new PaymentFailedEvent
            {
                OrderId = message.Id,
                CorrelationId = context.CorrelationId.Value,
                Reason = "Total Price was zero or negative"
            };
            await _publishEndpoint.Publish(failedEvent);
            _logger.LogWarning("Payment Failed for Order Id : {@OrderId} and Correlation Id : {@CorrelationId}", message.Id, messag
        }
    }
}
```

What happens:

1. Consumes `OrderCreatedEvent`
2. Processes payment (simulated with `Task.Delay`)
3. If success → publishes `PaymentCompletedEvent`
4. If failure → publishes `PaymentFailedEvent`

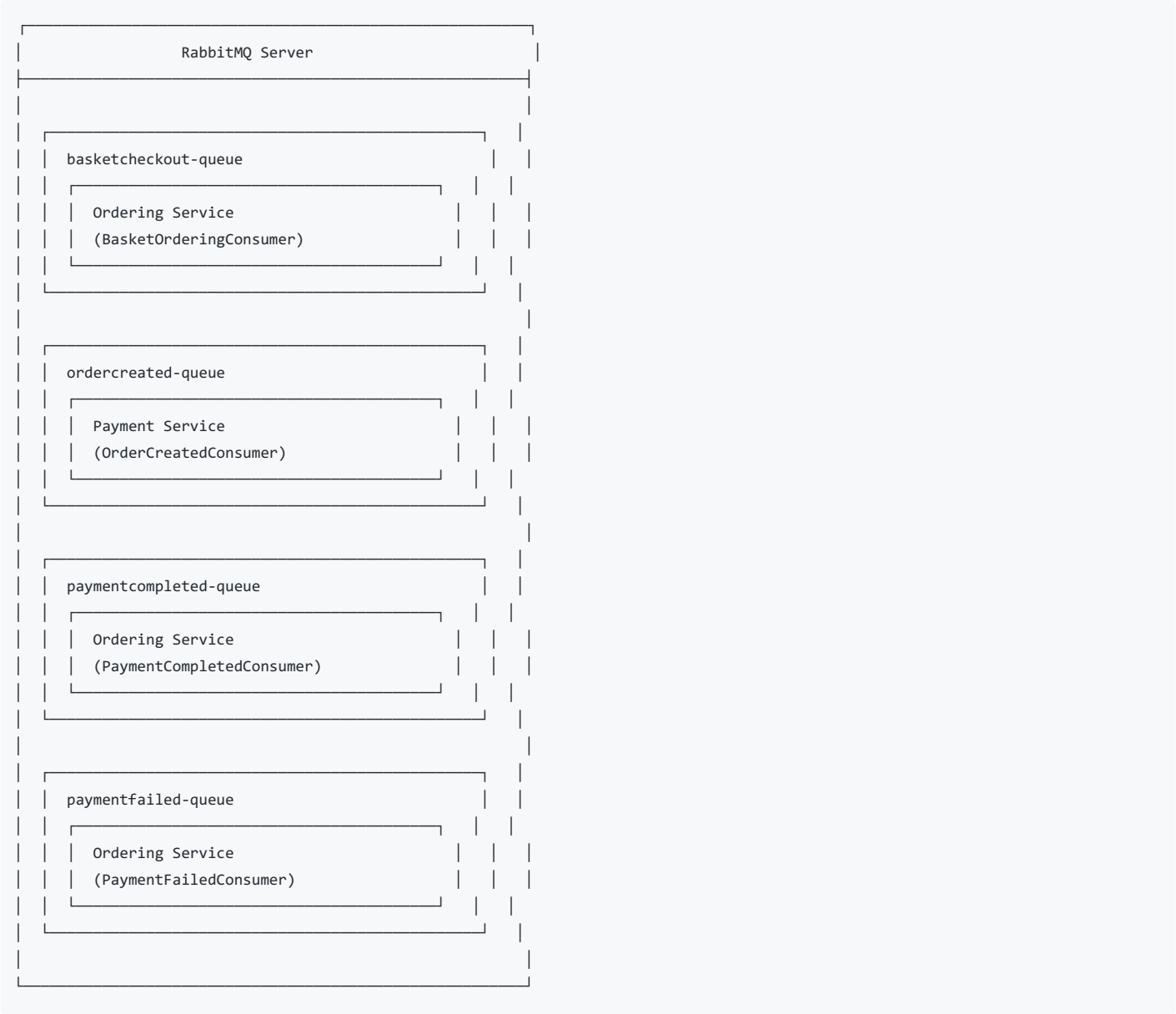
Step 5: Ordering Service Consumes Payment Result

File: `Ordering.Application/EventBusConsumer/PaymentCompletedConsumer.cs` (follows same pattern)

This consumer would update the order status in the database based on payment result.

Queue Architecture

RabbitMQ Queue Structure



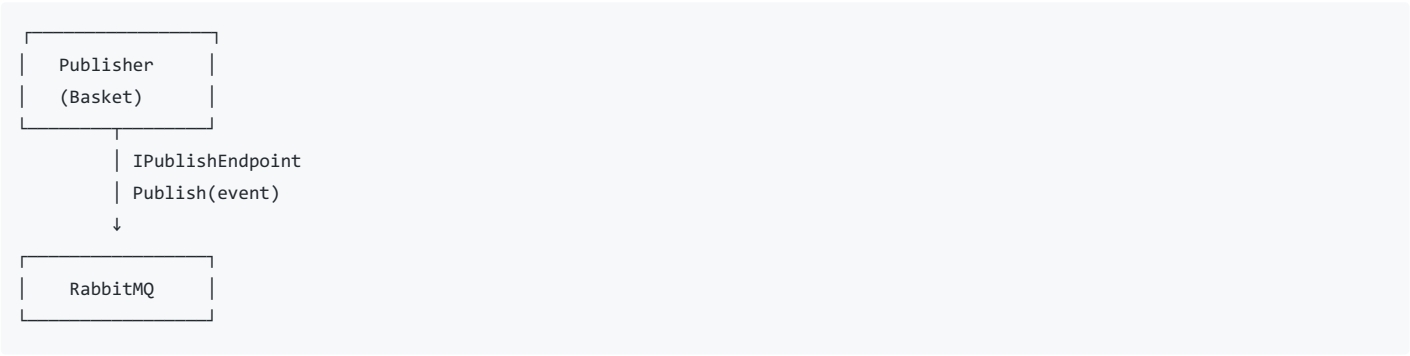
Queue-Consumer Mapping Table

Queue Name	Consumer	Service	Purpose
basketcheckout-queue	BasketOrderingConsumer	Ordering	Consumes basket checkout events
ordercreated-queue	OrderCreatedConsumer	Payment	Consumes order created events
paymentcompleted-queue	PaymentCompletedConsumer	Ordering	Consumes payment success events
paymentfailed-queue	PaymentFailedConsumer	Ordering	Consumes payment failure events

Key Concepts

Publisher vs Consumer

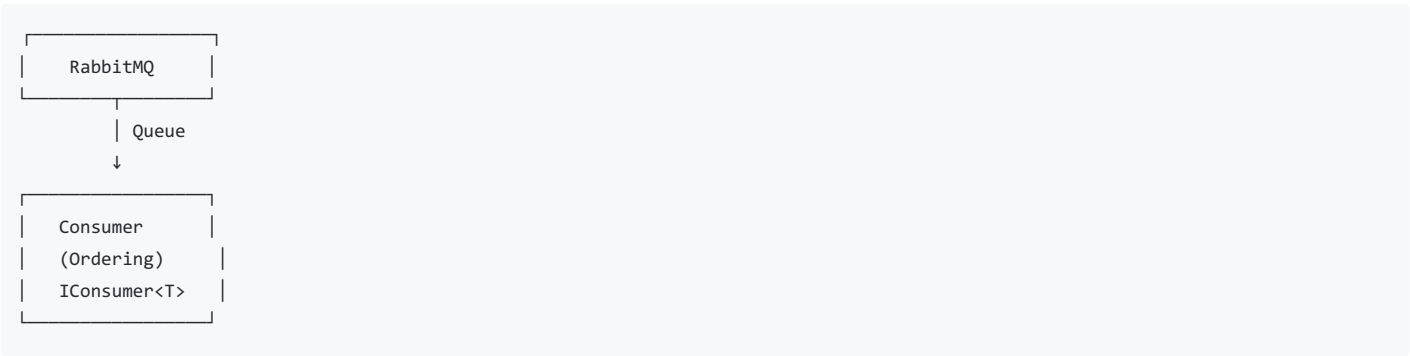
Publisher Pattern



Publisher (Basket Service):

- Sends messages to RabbitMQ
- Uses IPublishEndpoint
- Example: `await _publishEndpoint.Publish(event)`

Consumer Pattern

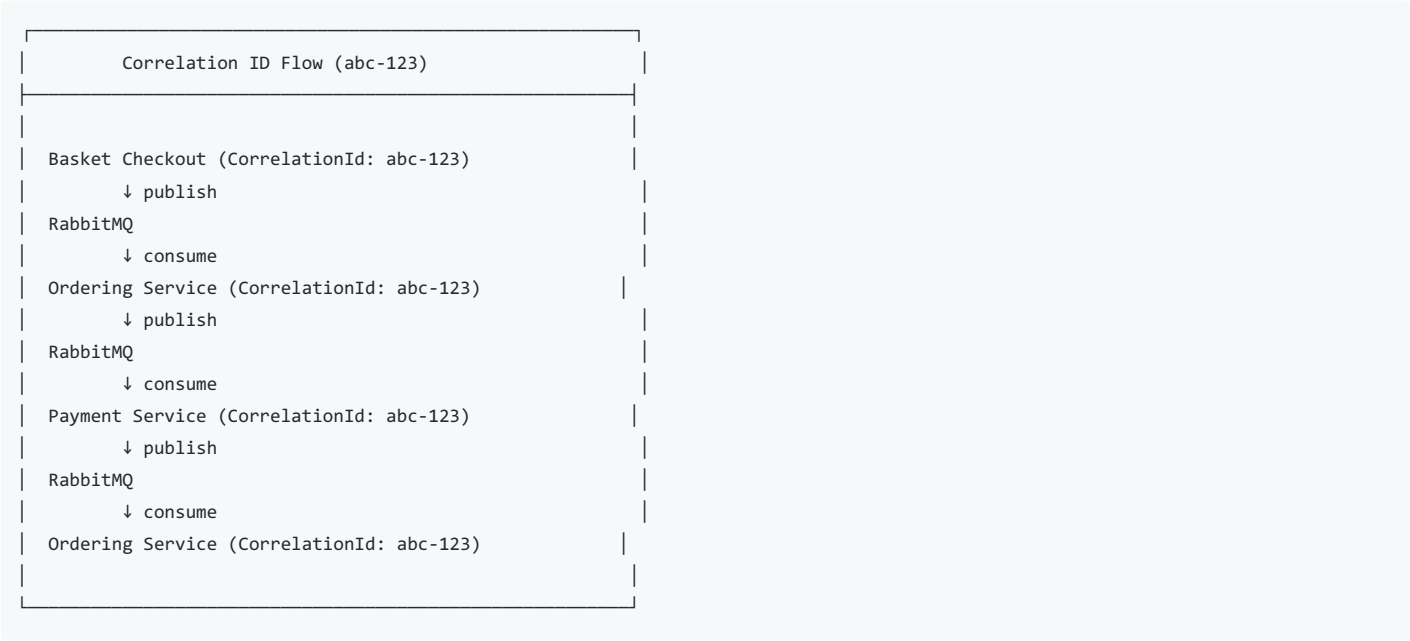


Consumer (Ordering, Payment Services):

- Receives messages from RabbitMQ
- Implements IConsumer<T>
- Example: `public class BasketOrderingConsumer : IConsumer<BasketCheckoutEvent>`

Correlation ID Tracking

Purpose: Track a request across multiple services

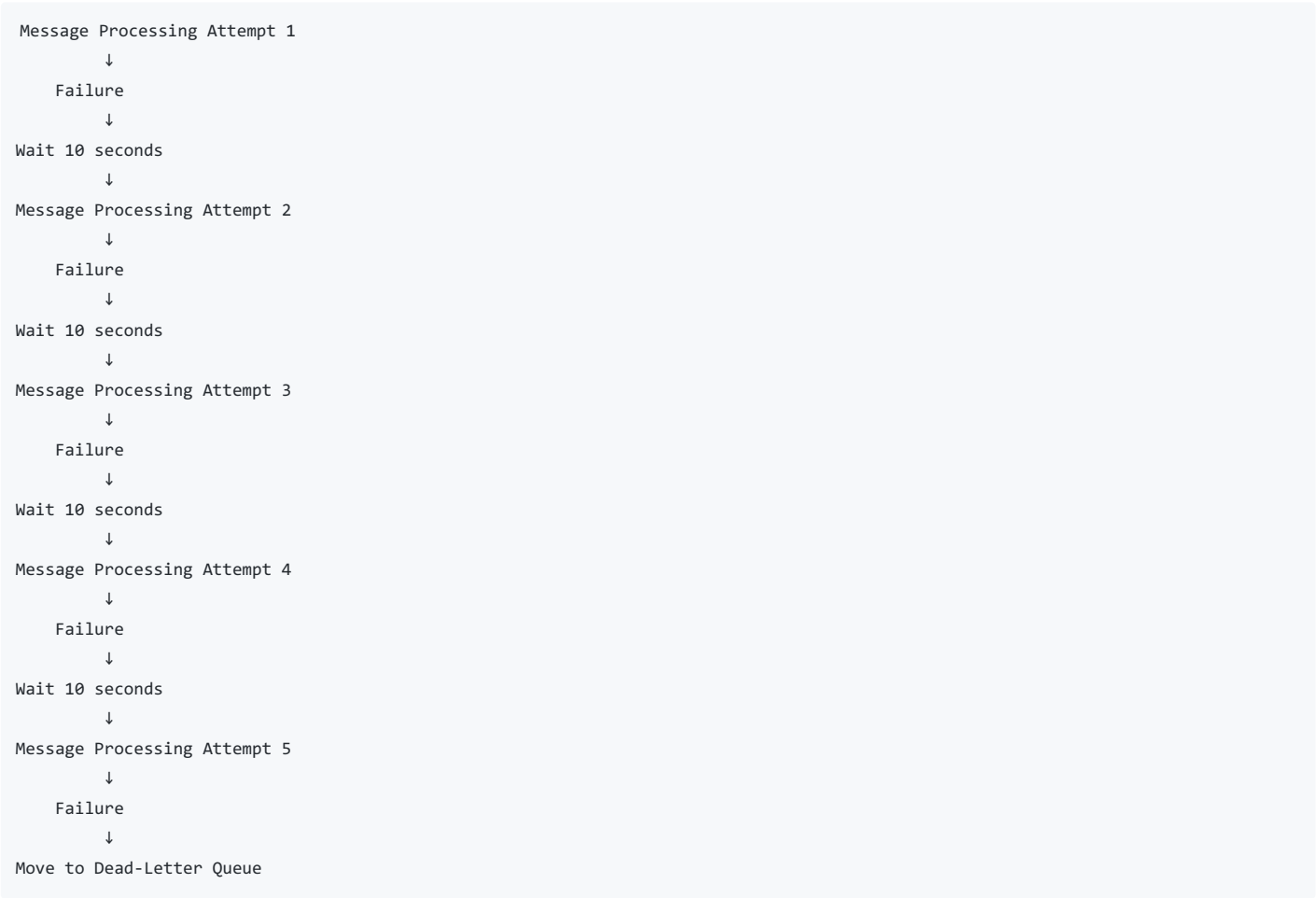


All events for one checkout have the same CorrelationId, making it easy to trace the entire flow.

Retry Policy

Your Configuration: `cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)))`

Retry Flow Diagram:



What it does:

- If message processing fails, retry up to 5 times
- Wait 10 seconds between retries
- After 5 failures, move message to dead-letter queue

Why this is important:

- Temporary failures (database timeout, network glitch) don't cause message loss
- Gives services time to recover
- Dead-letter queue for manual inspection of failed messages

Summary

RabbitMQ in Your Project

Your ECommerce microservices use RabbitMQ as a message broker to enable asynchronous, event-driven communication between services.

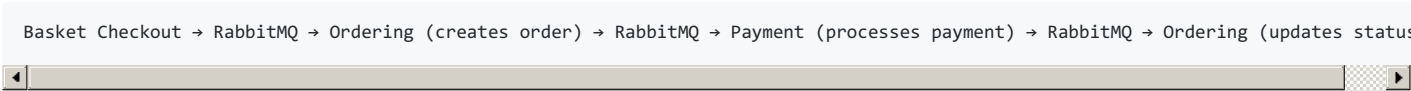
Implementation Overview

Implementation Summary
RabbitMQ Server: - Configured in docker-compose.yml (lines 17-18) - Includes management UI - Ports: 5672 (AMQP), 15672 (Management UI) Connection String: - amqp://guest:guest@rabbitmq:5672/ Libraries: - MassTransit 8.5.2 - MassTransit.RabbitMQ 8.5.2

Services Using RabbitMQ

Service	Role	Events
Basket	Publisher	Sends <code>BasketCheckoutEvent</code>
Ordering	Consumer + Publisher	Consumes <code>BasketCheckoutEvent</code> , <code>PaymentCompletedEvent</code> , <code>PaymentFailedEvent</code> ; Publishes <code>OrderCreatedEvent</code>
Payment	Consumer + Publisher	Consumes <code>OrderCreatedEvent</code> ; Publishes <code>PaymentCompletedEvent</code> , <code>PaymentFailedEvent</code>

Event Flow Summary



Key Features Implemented

- ❑ Queue definitions in `EventBusConstant.cs`
- ❑ Retry policy (5 retries, 10-second intervals)
- ❑ Correlation ID for request tracking
- ❑ Separate queues for each event type
- ❑ Proper error handling with dead-letter queue
- ❑ Asynchronous communication between services

Architecture Benefits

Your implementation follows best practices for event-driven microservices architecture with:

- **Decoupling:** Services don't directly depend on each other
- **Reliability:** Messages are not lost if services are down
- **Scalability:** Multiple consumers can process messages
- **Resilience:** Retry policies handle temporary failures
- **Observability:** Correlation IDs enable request tracing

Additional Resources

RabbitMQ Management UI

- URL: <http://localhost:15672>
- Username: guest
- Password: guest

Useful Commands

```
# Start RabbitMQ
docker-compose up -d rabbitmq

# View RabbitMQ logs
docker logs rabbitmq

# Restart RabbitMQ
docker-compose restart rabbitmq

# Stop RabbitMQ
docker-compose stop rabbitmq
```

Monitoring

In the RabbitMQ Management UI, you can:

- View all queues and their message counts
- Monitor message rates
- View connections and channels
- Inspect dead-letter queues
- Trace message flow with correlation IDs

This guide provides a complete overview of RabbitMQ and MassTransit implementation in your ECommerce microservices project.